Working with Arrays

CS300: Programming II

Cole Nelson

Learning Objectives

1. Be able to **explain** primitives vs. references.
2. Be able to **distinguish** `{}` from `new int[n]`.
3. Be able to **understand** oversize `add` / `remove` / `search`.
4. Be able to **use** 2D and parallel arrays.
5. Be able to **write** a tester suite.

Where this fits

Last Lecture: Perfect size vs oversize arrays.

- See Hobbes' definition! Including "compact".

Everything today is a **visualization** – watch, trace, predict. At the end, we'll show Eclipse and submitting to GradeScope.

What does a variable *hold*?

`int` and `int[]` behave differently when you copy them. Predict both printouts, then we'll watch it run.

→ Step through it in Java Visualizer.

A primitive holds its value

```
int x = 5;
int y = x;   // y gets a COPY of 5
y = 99;
// x is still 5
```

`y = x` copies the **value**. `x` and `y` are two separate boxes — changing one can't touch the other.

An array holds a reference

```
int[] a = {1, 2, 3};
int[] b = a;    // copies the ARROW, not the data
b[0] = 99;
// a[0] is now 99, too
```

`b = a` copies the **reference** — both names point at the *same* array. This sharing is called **aliasing**.

Strings are references too: `==` compares references, not characters — use `.equals()` or `.equalsIgnoreCase()` to compare their contents.

Tophat Q1

After this runs, what is printed?

```
public static void bump(int[] arr) {
  arr[0] += 10;
}

public static void main(String[] args) {
  int[] nums = {1, 2, 3};
  bump(nums);
  System.out.println(nums[0]);
}
```

→ Visualize *after* submitting your vote.

`{}` vs `new Type[n]`

Three ways to make an array... What's the difference?

```
int[] a = {1, 2, 3};              // initializer – values you know now
int[] b = new int[3];             // sized, filled with defaults (0, 0, 0)
int[] c = new int[]{1, 2, 3};     // initializer WITH explicit new
```

→ Step through it in Java Visualizer.

Oversized array? You'll *always* need a `size` variable!

(We've been calling it `size`, call it what you will!)

Four oversize array operations

Every operation is driven by a `size`, not `length`; and we must keep it updated!

- `void print(String[] arr, int size)`
- `int add(String[] arr, int size, String val)`
- `int remove(String[] arr, int size, String val)`
- `int indexOf(String[] arr, int size, String val)`

You'll implement something like this using a 2d array in **P1** – use **zyBooks 1.7** as reference.

2D arrays

A 2D array is an array whose elements are themselves arrays – a grid of rows and columns.

→ Step through it in Java Visualizer.

A grid of rows and columns

```
int[][] grid = { {3, 8, 1},
                 {9, 2, 7} };
```

`grid` holds two rows; each row is an `int[]`. `grid[1]` is the whole row `{9, 2, 7}`.

Index twice to reach a value

Expression	Value
`grid[1]`	`{9, 2, 7}` — a whole row
`grid[1][0]`	`9` — row 1, column 0
`grid[1][2]`	`7` — row 1, column 2

In P1: the cart is a `String[][]`, so a count like `"2"` is *text* — call `Integer.parseInt(...)` before doing math.

Parallel arrays

Arrays that are added to and removed from in parallel.

```
String[] menu   = {"latte", "drip", "cocoa"};
int[]    prices = {   5,       3,      4    };
```

`menu[i]` costs `prices[i]`.

→ Step through it in Java Visualizer.

Testing

In CS400, you'll learn about a framework called **JUnit**.

In CS300, we simply write a "tester" file.

Let's test our `CoffeeShop` example!

→ Step through it in Java Visualizer.

Tophat Q2

You write the following test to be sure the price of a coconut is $3. The test fails; why?

```
public static boolean testPriceOf() {
    String[] menu   = {"apple", "banana", "coconut"};
    int[] prices = { 2, 1, 3 };

    if (CoffeeShop.priceOf(menu, prices, "coconut") == 3) {
      return false;
    } else {
      return true;
    }
}
```

Eclipse & P1

Let's walk through the process of using Eclipse and submitting P1.

WES-CS

Learn more!